

RAPPORT DE STAGE

BTS Services Informatiques aux Organisations
Option SLAM - Solutions Logicielles et Applications Métiers

CRPN

Caisse de Retraite du Personnel Navigant

*Développement de l'application **Contrathèque***

Présenté par : Yacine TOUAZI

Établissement : EPSI

Entreprise d'accueil : CRPN - Caisse de Retraite du Personnel Navigant

Durée du stage : 5 semaines

Année scolaire : 2025 - 2026

Remerciements

Avant d'entamer la présentation de ce rapport de stage, je souhaite adresser mes remerciements les plus sincères aux personnes qui m'ont apporté leur aide et qui ont contribué à l'élaboration de ce travail ainsi qu'à la réussite de cette expérience professionnelle particulièrement enrichissante.

Je tiens tout d'abord à remercier la **Caisse de Retraite du Personnel Navigant (CRPN)** pour m'avoir accueilli au sein de ses équipes durant cette période de cinq semaines. Cette immersion dans le monde professionnel m'a permis de découvrir un environnement de travail structuré et d'appréhender concrètement les exigences du métier de développeur logiciel.

Je remercie tout particulièrement **mon maître de stage** pour son accompagnement constant, sa disponibilité, sa pédagogie et la confiance qu'il m'a accordée tout au long de cette mission. Ses conseils avisés et son expertise technique m'ont été d'une aide précieuse dans la conception et le développement de l'application Contrathèque.

Mes remerciements s'adressent également à **l'ensemble du service informatique** de la CRPN pour leur accueil chaleureux, leur patience et leur volonté de partager leurs connaissances. Leurs explications sur le fonctionnement du système d'information et leurs retours d'expérience ont grandement contribué à mon apprentissage.

Je souhaite également exprimer ma reconnaissance à **l'équipe pédagogique de l'EPSI**, et plus particulièrement à mes professeurs du BTS SIO option SLAM, pour la qualité de leur enseignement et pour m'avoir préparé à affronter le monde professionnel. Les compétences techniques et méthodologiques acquises durant ma formation m'ont permis d'aborder ce stage avec sérénité et confiance.

Enfin, je tiens à remercier ma famille et mes proches pour leur soutien indéfectible tout au long de mon parcours scolaire et professionnel. Leur présence et leurs encouragements constituent une source de motivation essentielle.

Sommaire

Remerciements	2
Sommaire	3
Introduction	4
I. Présentation de l'entreprise	5
1.1 Historique et missions de la CRPN	5
1.2 Organisation et structure	6
1.3 Le service informatique	7
1.4 L'environnement technique	8
II. Le projet : Contrathèque	9
2.1 Contexte et enjeux	9
2.2 Cahier des charges	10
2.3 Analyse des besoins	11
2.4 Choix techniques	12
III. Conception et réalisation	13
3.1 Architecture de l'application	13
3.2 Modélisation de la base de données	14
3.3 Développement backend (Python)	16
3.4 Interface utilisateur	18
3.5 Tests et validation	19
IV. Bilan du stage	21
4.1 Compétences techniques acquises	21

4.2 Compétences professionnelles	22
4.3 Difficultés rencontrées	23
4.4 Apports personnels	23
Conclusion	24
Glossaire et annexes	25

Introduction

Dans le cadre de ma deuxième année de **BTS Services Informatiques aux Organisations** (SIO) option **Solutions Logicielles et Applications Métiers** (SLAM) à l'**EPSI**, j'ai eu l'opportunité d'effectuer un stage de cinq semaines au sein de la **Caisse de Retraite du Personnel Navigant (CRPN)**. Cette période d'immersion professionnelle constitue une étape essentielle de ma formation, me permettant de mettre en application les connaissances théoriques acquises tout au long de mon cursus.

Ce stage s'inscrit dans une démarche de professionnalisation et a pour objectif principal de me confronter aux réalités du métier de développeur logiciel dans un environnement structuré et exigeant. La CRPN, en tant qu'organisme de protection sociale dédié au personnel navigant de l'aéronautique civile, dispose d'un système d'information complexe qui nécessite un maintien constant et des évolutions régulières pour répondre aux besoins métiers en perpétuelle évolution.

Au cours de cette mission, j'ai été chargé du développement d'une application baptisée « **Contrathèque** ». Cet outil interne vise à **regrouper et centraliser l'ensemble des factures de l'entreprise sur une seule et même plateforme**, facilitant ainsi leur consultation, leur gestion et leur archivage. Ce projet, bien que circonscrit dans le temps, représente un véritable défi technique et organisationnel puisqu'il implique la maîtrise de plusieurs technologies (Python, MySQL) et une bonne compréhension de l'architecture du système d'information existant.

Ce rapport a pour vocation de restituer fidèlement l'expérience vécue durant ces cinq semaines de stage. Il s'articule autour de quatre grandes parties :

- Une première partie consacrée à la **présentation de l'entreprise d'accueil**, son histoire, ses missions, son organisation et son environnement technique.
- Une deuxième partie détaillant le **projet Contrathèque**, son contexte, ses enjeux, le cahier des charges et l'analyse des besoins.
- Une troisième partie axée sur la **conception et la réalisation technique** de l'application, depuis l'architecture jusqu'aux tests de validation.
- Enfin, une dernière partie présentant le **bilan** de cette expérience, tant sur le plan des compétences techniques acquises que sur l'aspect professionnel et personnel.

L'objectif de ce rapport est non seulement de rendre compte du travail accompli, mais également de proposer une réflexion critique sur les méthodes employées, les difficultés rencontrées et les enseignements tirés de cette première véritable expérience en entreprise.

I. Présentation de l'entreprise

1.1 Historique et missions de la CRPN

La **Caisse de Retraite du Personnel Navigant professionnel de l'aéronautique civile (CRPN)** est un organisme de droit privé chargé d'une mission de service public. Créée en 1951, elle assure la gestion du régime de retraite complémentaire obligatoire de l'ensemble du personnel navigant professionnel de l'aéronautique civile française, qu'il s'agisse du personnel navigant technique (pilotes, mécaniciens navigants) ou du personnel navigant commercial (hôtesses de l'air, stewards, chefs de cabine).

La CRPN intervient à plusieurs titres dans le quotidien de ses affiliés :

- **Collecte des cotisations** auprès des employeurs et des salariés du secteur aérien ;
- **Liquidation et versement des pensions** de retraite et d'invalidité ;
- **Gestion des dossiers individuels** des affiliés tout au long de leur carrière ;
- **Information et accompagnement** des cotisants et des retraités sur leurs droits ;
- **Gestion financière** du régime et placement des réserves.

Quelques chiffres clés

Indicateur	Valeur
Année de création	1951
Nombre d'affiliés actifs	Environ 45 000
Nombre de pensionnés	Plus de 35 000
Effectif de l'organisme	Environ 110 collaborateurs
Siège social	Paris
Statut juridique	Organisme de droit privé

Tableau 1 : Données clés de la CRPN

La CRPN est un acteur incontournable du paysage de la protection sociale aéronautique en France. Son rôle est d'autant plus important que le métier de personnel navigant présente des spécificités fortes (carrières souvent courtes, limites d'âge réglementaires, conditions de travail particulières) qui nécessitent un régime de retraite adapté.

1.2 Organisation et structure

La CRPN s'organise autour de plusieurs directions et services qui travaillent en synergie pour remplir les missions de service public confiées à l'organisme. La structure interne reflète la diversité des activités menées, de la gestion administrative aux opérations financières en passant par les fonctions support.

Les principales directions

L'organigramme général de la CRPN se compose des entités suivantes :

- **La Direction Générale** : pilote l'ensemble de l'organisme et définit la stratégie ;
- **La Direction des Retraites** : assure la gestion des dossiers d'affiliés et le versement des prestations ;
- **La Direction Financière et Comptable** : gère les flux financiers, le budget et la comptabilité ;
- **La Direction des Systèmes d'Information (DSI)** : conçoit, maintient et fait évoluer le système d'information ;
- **La Direction des Ressources Humaines** : gère le personnel interne de la CRPN ;
- **Les services support** (juridique, communication, achats, logistique).

Gouvernance et instances

La CRPN est administrée par un **Conseil d'Administration paritaire** composé de représentants des employeurs et des salariés du secteur. Ce conseil définit les orientations générales, vote le budget et contrôle l'exécution des missions. Plusieurs commissions spécialisées (commission financière, commission technique, commission de recours amiable) traitent des sujets spécifiques et préparent les décisions du Conseil.

Cette gouvernance paritaire garantit la prise en compte équilibrée des intérêts de l'ensemble des parties prenantes du régime. Elle s'inscrit dans la tradition française de gestion paritaire des organismes de protection sociale.

Implantation géographique

Le siège social de la CRPN est situé à Paris, dans des locaux modernes qui regroupent l'ensemble des services. L'organisme ne dispose pas d'antennes régionales, mais s'appuie sur un réseau de partenariats avec d'autres organismes sociaux et avec les employeurs du secteur pour assurer la proximité avec les affiliés.

L'organisation du travail est mixte, combinant présence sur site et télétravail. Cette modalité hybride a été pérennisée après la crise sanitaire et favorise la flexibilité tout en maintenant la cohésion d'équipe.

1.3 Le service informatique

La **Direction des Systèmes d'Information (DSI)** de la CRPN constitue l'environnement direct dans lequel s'est déroulé mon stage. C'est au sein de cette direction que j'ai été intégré pour mener à bien ma mission de développement.

Organisation de la DSI

La DSI de la CRPN se structure autour de plusieurs pôles complémentaires qui travaillent en étroite collaboration :

- **Le pôle Études et Développement** : responsable de la conception et du développement des applications internes. C'est dans ce pôle que j'ai été affecté durant mon stage. Les développeurs y travaillent sur l'évolution des outils métiers, l'intégration de nouvelles fonctionnalités et la maintenance corrective et évolutive du parc applicatif.
- **Le pôle Production et Exploitation** : assure le maintien en condition opérationnelle des systèmes en production, la supervision des traitements, la gestion des incidents et la continuité de service.
- **Le pôle Infrastructure et Réseaux** : gère les serveurs, les équipements réseau, la sécurité informatique, les sauvegardes et l'ensemble de l'infrastructure technique.
- **Le pôle Support utilisateurs** : accompagne les collaborateurs internes dans l'utilisation quotidienne des outils et résout les incidents de niveau 1 et 2.

Méthodologie de travail

Le service informatique de la CRPN s'appuie sur des **méthodologies de gestion de projet modernes** inspirées des principes agiles. Les développements suivent un cycle structuré qui comprend les phases d'analyse des besoins, de conception, de développement, de tests, de recette et de mise en production.

Des **réunions hebdomadaires** rassemblent l'équipe de développement pour faire le point sur l'avancement des projets, identifier les blocages éventuels et planifier les tâches à venir. Cette communication régulière favorise la cohésion d'équipe et permet une meilleure coordination des travaux. J'ai eu l'opportunité de participer à plusieurs de ces réunions, ce qui m'a permis de comprendre la dynamique collective et les enjeux transverses des projets.

Outils collaboratifs et de gestion

La DSI utilise un ensemble d'outils dédiés à la gestion de projet et à la collaboration :

- Un **gestionnaire de tickets** pour le suivi des demandes, des bugs et des évolutions ;
- Un **système de gestion de versions** (Git) pour le suivi du code source ;
- Une **plateforme de documentation interne** centralisant les procédures techniques ;
- Des outils de **communication instantanée** pour les échanges au quotidien.

1.4 L'environnement technique

L'environnement technique de la CRPN est riche et diversifié. Il combine des technologies historiques, héritées de plusieurs décennies d'évolution du système d'information, et des technologies plus modernes utilisées pour les nouveaux développements. Cette hétérogénéité constitue à la fois une richesse et un défi pour la DSI.

Le parc applicatif

Le système d'information de la CRPN se compose de plusieurs applications métiers spécialisées qui couvrent l'ensemble des processus de l'organisme : gestion des affiliés, calcul des droits, liquidation des pensions, gestion comptable, déclarations légales, etc. Ces applications sont interconnectées et échangent quotidiennement de nombreuses données.

Le service informatique veille à la cohérence d'ensemble de ce parc applicatif et à la fiabilité des échanges entre les différents systèmes. La qualité des données est une préoccupation majeure, compte tenu des enjeux financiers et juridiques qui y sont associés.

Technologies utilisées

Catégorie	Technologies
Langages de programmation	Python, Java, JavaScript, SQL
Bases de données	MySQL, Oracle, PostgreSQL
Systèmes d'exploitation	Linux (Debian, RedHat), Windows Server
Serveurs d'application	Apache, Nginx, Tomcat
Gestion de versions	Git (GitLab interne)
Frameworks web	Flask, Django (côté Python)
Outils de tests	PyTest, Postman
Documentation	Markdown, Confluence-like

Tableau 2 : Stack technique de la CRPN

Pour mon projet de Contrathèque, j'ai principalement travaillé avec **Python** comme langage principal et **MySQL** comme système de gestion de base de données. Ce choix technologique s'inscrit dans la cohérence des outils déjà déployés au sein de la CRPN et bénéficie d'un écosystème mature et bien documenté.

Politique de sécurité

La CRPN traite des données à caractère personnel et financier extrêmement sensibles. La **sécurité informatique** est par conséquent un enjeu majeur. L'organisme applique

rigoureusement les exigences du **Règlement Général sur la Protection des Données (RGPD)** et met en œuvre des mesures techniques et organisationnelles pour garantir la confidentialité, l'intégrité et la disponibilité des données.

Tout au long de mon stage, j'ai été sensibilisé à ces problématiques et j'ai intégré les bonnes pratiques de sécurité dans mes développements (gestion des accès, chiffrement, validation des entrées, journalisation, etc.).

II. Le projet : Contrathèque

2.1 Contexte et enjeux

Le projet qui m'a été confié durant mon stage porte le nom de « **Contrathèque** ». Il s'agit d'une application interne destinée à centraliser et à gérer l'ensemble des factures de la CRPN sur une plateforme unique. Avant ce projet, la gestion des factures était dispersée entre plusieurs outils et supports (tableurs, dossiers réseau, applications métiers diverses), ce qui rendait difficile le suivi global et générait des inefficacités.

Genèse du projet

Le besoin d'une solution centralisée s'est manifesté progressivement, à mesure que le volume de documents traités par la CRPN augmentait. Les utilisateurs métiers (service comptable, service achats, contrôle de gestion) exprimaient de plus en plus fréquemment le besoin d'**accéder rapidement aux factures**, de pouvoir **effectuer des recherches multicritères** et de **générer des rapports synthétiques**.

Par ailleurs, le cadre réglementaire impose des exigences strictes en matière de conservation des documents comptables et fiscaux. Une solution unifiée présente l'avantage de garantir le respect de ces obligations tout en simplifiant les processus internes.

Enjeux du projet

Les enjeux de la Contrathèque sont multiples et s'articulent autour de plusieurs axes :

- **Enjeu fonctionnel** : offrir aux utilisateurs une interface ergonomique permettant de consulter, rechercher, ajouter et gérer les factures avec efficacité.
- **Enjeu organisationnel** : harmoniser les pratiques entre les services, fluidifier les processus de validation et améliorer la traçabilité des opérations.
- **Enjeu technique** : concevoir une application robuste, maintenable et évolutive, respectant les standards techniques de la CRPN.
- **Enjeu de sécurité** : protéger les données sensibles contenues dans les factures (montants, fournisseurs, références) et assurer leur intégrité dans la durée.
- **Enjeu réglementaire** : garantir le respect des obligations légales de conservation et de traçabilité des documents comptables.

Mon positionnement dans le projet

En tant que stagiaire développeur, j'ai été pleinement intégré à l'équipe en charge du projet. Mon rôle a consisté à participer activement à toutes les phases : analyse des besoins, conception, développement, tests et documentation. Cette approche complète m'a permis d'appréhender l'ensemble du cycle de vie d'un projet logiciel et de comprendre les interactions entre les différents acteurs.

2.2 Cahier des charges

Le cahier des charges du projet Contrathèque a été élaboré en amont de mon arrivée, en concertation avec les utilisateurs métiers. Il définit précisément les fonctionnalités attendues, les contraintes techniques et les critères de réussite du projet. Sa lecture et son analyse ont constitué l'une de mes premières activités au sein de la CRPN.

Objectifs fonctionnels

L'application Contrathèque doit permettre aux utilisateurs autorisés d'effectuer les opérations suivantes :

Catégorie	Fonctionnalités attendues
Consultation	Visualiser la liste des factures, filtrer par fournisseur, par date, par montant
Ajout	Saisir une nouvelle facture, joindre un document numérisé (PDF, image)
Modification	Mettre à jour les informations d'une facture existante
Suppression	Archiver ou supprimer une facture selon les droits
Recherche	Effectuer des recherches multicritères avancées
Export	Exporter des données au format CSV ou Excel
Statistiques	Visualiser des tableaux de bord avec indicateurs clés
Historique	Consulter le journal des modifications de chaque facture

Tableau 3 : Fonctionnalités principales de la Contrathèque

Contraintes techniques

Le cahier des charges impose plusieurs contraintes techniques qui ont orienté les choix de conception :

- L'application doit être **développée en Python**, langage déjà utilisé pour d'autres outils internes de la CRPN ;
- La base de données doit s'appuyer sur **MySQL**, en cohérence avec les pratiques de l'organisme ;
- L'application doit être **compatible avec l'architecture du système d'information** existant et pouvoir s'intégrer aux autres outils internes ;
- Les **performances** doivent être satisfaisantes même avec un volume important de factures (plusieurs dizaines de milliers d'enregistrements) ;
- La **sécurité** des données doit être garantie à tous les niveaux (authentification, autorisations, chiffrement, journalisation).

2.3 Analyse des besoins

L'analyse des besoins a constitué une étape déterminante du projet. Elle m'a permis de mieux comprendre les attentes des utilisateurs et de traduire ces attentes en spécifications techniques exploitables pour le développement. Cette phase s'est déroulée en plusieurs temps et a impliqué différents acteurs.

Identification des utilisateurs cibles

L'application Contrathèque s'adresse principalement à trois profils d'utilisateurs au sein de la CRPN, chacun ayant des besoins spécifiques :

- **Les comptables** : utilisent quotidiennement l'application pour saisir, modifier et consulter les factures dans le cadre de la gestion comptable. Ils ont besoin d'un outil rapide, précis et fiable.
- **Les responsables d'achats** : consultent les factures pour suivre les engagements et négocier avec les fournisseurs. Ils privilégient les fonctionnalités de recherche et de filtrage.
- **Le contrôle de gestion** : analyse les données pour produire des indicateurs et des reportings. Cette population a besoin d'exports et de tableaux de bord synthétiques.

Les **administrateurs techniques** constituent une catégorie distincte d'utilisateurs, responsables de la configuration de l'application, de la gestion des droits et de la maintenance.

Recueil des besoins fonctionnels

Plusieurs **entretiens** ont été menés avec des représentants des services concernés afin de cerner précisément leurs attentes. J'ai eu l'opportunité de participer à certains de ces échanges, ce qui a constitué une expérience particulièrement formatrice. Comprendre le métier de ses utilisateurs est en effet une compétence essentielle pour un développeur, parfois sous-estimée mais cruciale pour la réussite d'un projet.

Les besoins recueillis ont été regroupés et hiérarchisés selon leur importance (essentiel, important, souhaitable) et leur complexité de mise en œuvre. Cette priorisation a permis de planifier les développements de manière cohérente et de garantir la livraison d'une première version fonctionnelle dans les délais impartis.

Cas d'utilisation

L'analyse des besoins a débouché sur l'identification d'un ensemble de **cas d'utilisation** (*use cases*) qui décrivent les interactions typiques entre les utilisateurs et l'application. Parmi les principaux cas d'utilisation, on peut citer :

- Saisir une nouvelle facture suite à sa réception ;
- Rechercher une facture par numéro, fournisseur ou date ;
- Modifier le statut d'une facture (en attente, validée, payée, contestée) ;
- Générer un rapport mensuel des factures par fournisseur ;
- Exporter les données vers un tableur pour analyse complémentaire.

2.4 Choix techniques

Les choix techniques du projet ont été en grande partie déterminés par le cadre imposé par le cahier des charges et par l'existant technique de la CRPN. Cependant, plusieurs décisions complémentaires ont dû être prises pour structurer le projet et optimiser sa réalisation.

Le langage Python

Python a été retenu comme langage principal pour plusieurs raisons. C'est tout d'abord un langage **moderne, polyvalent et lisible**, dont la syntaxe favorise la productivité et la maintenabilité du code. Il bénéficie d'un **écosystème extrêmement riche** avec de nombreuses bibliothèques tierces couvrant tous les besoins courants : accès aux bases de données, génération de PDF, manipulation de fichiers Excel, interfaces graphiques, etc.

Par ailleurs, Python est déjà **présent dans le parc applicatif de la CRPN**, ce qui facilite l'intégration et la maintenance ultérieure du projet par les équipes en place. La courbe d'apprentissage de ce langage est relativement douce, ce qui constitue un atout supplémentaire pour la pérennité de l'application.

Le SGBD MySQL

MySQL a été choisi comme système de gestion de base de données. Il s'agit d'un **SGBD relationnel** mature, performant et largement éprouvé. Ses principaux atouts dans le contexte du projet sont les suivants :

- **Fiabilité** : MySQL est utilisé par des millions d'applications dans le monde et offre un niveau de stabilité particulièrement élevé ;
- **Performance** : il est capable de gérer efficacement de gros volumes de données et offre de bonnes performances en lecture comme en écriture ;
- **Compatibilité** : il s'intègre parfaitement avec Python via des connecteurs standards comme *mysql-connector-python* ou *PyMySQL* ;
- **Sécurité** : il propose des mécanismes robustes de gestion des droits et de chiffrement des données.

Architecture du Système d'Information

Une attention particulière a été portée à **l'architecture du SI**. La Contrathèque n'est pas un système isolé : elle doit pouvoir communiquer avec d'autres composants du système d'information de la CRPN. Cette dimension architecturale a guidé plusieurs choix de conception, notamment l'organisation en couches (présentation, métier, données) et l'utilisation d'un format d'échange standardisé pour les interactions avec les autres applications.

Le choix d'une **architecture modulaire** facilite la maintenance et les évolutions futures. Chaque composant peut être modifié indépendamment des autres, à condition que les interfaces (contrats) soient respectées. Cette approche s'inscrit dans les bonnes pratiques modernes du développement logiciel et garantit la pérennité de la solution.

III. Conception et réalisation

3.1 Architecture de l'application

L'architecture logicielle de la Contrathèque a été conçue selon le modèle **MVC (Model-View-Controller)**, un patron de conception largement adopté dans le développement d'applications modernes. Ce modèle permet une séparation claire des responsabilités et favorise la maintenabilité du code.

Le patron MVC

Le modèle MVC structure une application en trois couches distinctes :

- **Le Modèle (Model)** : représente les données de l'application et la logique métier. Dans le cas de la Contrathèque, il s'agit des entités comme « Facture », « Fournisseur », « Utilisateur », ainsi que des règles de gestion associées (calcul de TVA, validation de montants, gestion des statuts, etc.).
- **La Vue (View)** : correspond à l'interface utilisateur, c'est-à-dire ce que voit et manipule l'utilisateur final. Elle est chargée d'afficher les informations transmises par le contrôleur et de recueillir les actions de l'utilisateur.
- **Le Contrôleur (Controller)** : fait le lien entre le modèle et la vue. Il reçoit les actions de l'utilisateur via la vue, déclenche les opérations sur le modèle et met à jour la vue en conséquence.

Cette séparation présente plusieurs avantages : elle facilite le développement en équipe (chaque développeur peut se concentrer sur une couche), elle améliore la testabilité du code (chaque composant peut être testé indépendamment) et elle simplifie l'évolution de l'application (modifier l'interface n'impacte pas la logique métier).

Structure des fichiers du projet

L'organisation des fichiers du projet reflète l'architecture choisie. Voici la structure type adoptée :

```
contratheque/
|-- main.py # Point d'entree
|-- config.py # Configuration
|-- requirements.txt # Dependances
|
|-- models/
|   |-- facture.py
|   |-- fournisseur.py
|   |-- utilisateur.py
|
|-- controllers/
|   |-- facture_controller.py
|   |-- auth_controller.py
|
|-- views/
|   |-- main_view.py
|   |-- facture_view.py
|
```

```
|-- database/  
| |-- connection.py  
| `-- queries.py  
|  
|-- utils/  
| |-- validators.py  
| `-- exporters.py
```

Cette organisation modulaire permet à chaque développeur de retrouver rapidement les éléments dont il a besoin et favorise une bonne séparation des préoccupations.

3.2 Modélisation de la base de données

La modélisation de la base de données est une étape critique dans la conception d'une application centrée sur les données comme la Contrathèque. Une base bien modélisée garantit l'intégrité des informations, optimise les performances et facilite les évolutions futures. Cette phase a mobilisé une part importante de mon temps en début de projet.

Identification des entités

L'analyse fonctionnelle a permis d'identifier les principales entités à modéliser :

Entité	Description
Facture	Document comptable représentant une dépense engagée par la CRPN
Fournisseur	Entité commerciale ayant émis une facture
Utilisateur	Personne autorisée à se connecter et à utiliser l'application
Catégorie	Classification permettant de regrouper les factures par nature
Statut	État du cycle de vie d'une facture (en attente, validée, payée...)
Document	Pièce jointe associée à une facture (PDF scanné, image)
Historique	Trace des modifications successives apportées aux factures

Tableau 4 : Principales entités de la base de données

Relations entre les entités

Les entités identifiées entretiennent des relations qu'il convient de modéliser avec précision :

- Une **facture** est émise par **un fournisseur** (relation N-1).
- Une **facture** appartient à **une catégorie** (relation N-1).
- Une **facture** a **un statut** à un instant donné (relation N-1).
- Une **facture** peut être associée à plusieurs **documents** (relation 1-N).
- Une **facture** possède un **historique** de modifications (relation 1-N).
- Un **utilisateur** peut créer ou modifier plusieurs **factures** (relation 1-N).

Schéma de la table Facture

À titre d'exemple, voici la structure de la table principale **Facture**, qui constitue le cœur de l'application :

Champ	Type	Description
id_facture	INT (PK)	Identifiant unique
numero_facture	VARCHAR(50)	Référence du fournisseur
id_fournisseur	INT (FK)	Clé étrangère vers Fournisseur
date_emission	DATE	Date d'émission de la facture
date_echeance	DATE	Date d'échéance de paiement
montant_ht	DECIMAL(10, 2)	Montant hors taxes
taux_tva	DECIMAL(4, 2)	Taux de TVA appliqué
montant_ttc	DECIMAL(10, 2)	Montant toutes taxes comprises
id_categorie	INT (FK)	Clé étrangère vers Catégorie
id_statut	INT (FK)	Clé étrangère vers Statut
description	TEXT	Description libre
date_creation	DATETIME	Date de saisie dans le système
id_utilisateur	INT (FK)	Utilisateur ayant créé l'entrée

Tableau 5 : Structure de la table Facture

Normalisation et contraintes

La base de données a été conçue en respectant les **règles de normalisation** jusqu'à la **3^e forme normale**, ce qui garantit l'absence de redondance et facilite la maintenance. Les contraintes d'intégrité (clés primaires, clés étrangères, contraintes NOT NULL, contraintes UNIQUE, contraintes CHECK) ont été définies avec soin pour assurer la cohérence des données.

Des **index** ont été créés sur les colonnes fréquemment utilisées dans les recherches (numéro de facture, date d'émission, fournisseur) afin d'optimiser les performances. Cette optimisation est particulièrement importante compte tenu du volume de données attendu.

Création des tables en SQL

Voici un extrait du script SQL utilisé pour créer la table Facture :

```
CREATE TABLE Facture (
  id_facture INT AUTO_INCREMENT PRIMARY KEY,
  numero_facture VARCHAR(50) NOT NULL UNIQUE,
  id_fournisseur INT NOT NULL,
  date_emission DATE NOT NULL,
  date_echeance DATE,
```

```
montant_ht DECIMAL(10,2) NOT NULL,  
taux_tva DECIMAL(4,2) DEFAULT 20.00,  
montant_ttc DECIMAL(10,2) NOT NULL,  
id_categorie INT,  
id_statut INT NOT NULL DEFAULT 1,  
description TEXT,  
date_creation DATETIME DEFAULT CURRENT_TIMESTAMP,  
id_utilisateur INT NOT NULL,  
FOREIGN KEY (id_fournisseur) REFERENCES Fournisseur(id),  
FOREIGN KEY (id_categorie) REFERENCES Categorie(id),  
FOREIGN KEY (id_statut) REFERENCES Statut(id),  
FOREIGN KEY (id_utilisateur) REFERENCES Utilisateur(id),  
INDEX idx_numero (numero_facture),  
INDEX idx_date (date_emission)  
);
```

3.3 Développement backend (Python)

Le développement du backend a constitué la part la plus importante de mon stage. C'est à ce niveau que se concentre la logique métier de l'application : interaction avec la base de données, traitement des règles de gestion, exposition des fonctionnalités aux couches supérieures.

Organisation du code Python

Le code Python a été organisé en **modules** reflétant les différentes entités du système. Chaque entité dispose de sa propre classe regroupant les attributs et les méthodes associées. Cette approche **orientée objet** facilite la lisibilité, la réutilisation et la maintenance du code.

À titre d'illustration, voici une version simplifiée de la classe **Facture** :

```
class Facture:
    def __init__(self, id_facture=None, numero=None,
                  id_fournisseur=None, date_emission=None,
                  montant_ht=0.0, taux_tva=20.0):
        self.id_facture = id_facture
        self.numero = numero
        self.id_fournisseur = id_fournisseur
        self.date_emission = date_emission
        self.montant_ht = montant_ht
        self.traux_tva = traux_tva
        self.montant_ttc = self.calculer_ttc()

    def calculer_ttc(self):
        """Calcule le montant TTC à partir du HT et du taux de TVA."""
        return round(self.montant_ht * (1 + self.traux_tva / 100), 2)

    def valider(self):
        """Vérifie la cohérence des données de la facture."""
        if self.montant_ht < 0:
            raise ValueError("Le montant HT ne peut pas être négatif.")
        if not self.numero:
            raise ValueError("Le numéro de facture est obligatoire.")
        return True
```

Connexion à la base de données

L'accès à la base MySQL se fait via le connecteur officiel **mysql-connector-python**. Une classe dédiée gère l'établissement de la connexion, en s'appuyant sur les paramètres définis dans un fichier de configuration distinct. Cette approche permet de séparer les paramètres sensibles (mot de passe, hôte) du code source.

```
import mysql.connector
from config import DB_CONFIG

class DatabaseConnection:
    def __init__(self):
        self.connection = None

    def connect(self):
        try:
            self.connection = mysql.connector.connect(
                host=DB_CONFIG['host'],
```

```
        user=DB_CONFIG['user'],
        password=DB_CONFIG['password'],
        database=DB_CONFIG['database']
    )
    return self.connection
except mysql.connector.Error as err:
    print(f"Erreur de connexion : {err}")
    return None

def close(self):
    if self.connection and self.connection.is_connected():
        self.connection.close()
```

Opérations CRUD

L'application implémente l'ensemble des opérations **CRUD** (Create, Read, Update, Delete) sur les principales entités. Voici un exemple de méthode pour récupérer une facture à partir de son identifiant :

```
def get_facture_par_id(self, id_facture):
    """Récupère une facture par son identifiant."""
    connection = DatabaseConnection().connect()
    if not connection:
        return None

    cursor = connection.cursor(dictionary=True)
    query = "SELECT * FROM Facture WHERE id_facture = %s"

    try:
        cursor.execute(query, (id_facture,))
        result = cursor.fetchone()
        return result
    except mysql.connector.Error as err:
        print(f"Erreur SQL : {err}")
        return None
    finally:
        cursor.close()
        connection.close()
```

L'utilisation de **requêtes paramétrées** (avec le marqueur %s) est une bonne pratique essentielle pour se prémunir contre les attaques par **injection SQL**. Ce point a fait l'objet d'une attention particulière tout au long du développement.

Gestion des erreurs

La gestion robuste des erreurs constitue un aspect essentiel d'une application professionnelle. Toutes les opérations susceptibles d'échouer (connexion à la base, requêtes SQL, conversions de types, opérations sur les fichiers) sont encadrées par des blocs **try/except**. Les exceptions sont capturées et journalisées dans un fichier de log dédié, ce qui facilite le diagnostic ultérieur en cas de problème.

Un effort particulier a été porté sur la **qualité des messages d'erreur** remontés aux utilisateurs. Plutôt que d'afficher des messages techniques cryptiques, l'application présente des messages clairs et constructifs qui orientent l'utilisateur vers la résolution.

Recherche multicritères

L'une des fonctionnalités phares de la Contrathèque est la **recherche multicritères**, qui permet aux utilisateurs de filtrer les factures selon plusieurs paramètres simultanément. L'implémentation repose sur la construction dynamique de la requête SQL en fonction des critères fournis :

```
def rechercher_factures(self, criteres):
    """Recherche les factures selon plusieurs critères."""
    query = "SELECT * FROM Facture WHERE 1=1"
    params = []
```

```
if criteres.get('fournisseur'):
    query += " AND id_fournisseur = %s"
    params.append(criteres['fournisseur'])

if criteres.get('date_debut'):
    query += " AND date_emission >= %s"
    params.append(criteres['date_debut'])

if criteres.get('date_fin'):
    query += " AND date_emission <= %s"
    params.append(criteres['date_fin'])

if criteres.get('montant_min'):
    query += " AND montant_ttc >= %s"
    params.append(criteres['montant_min'])

cursor.execute(query, tuple(params))
return cursor.fetchall()
```

3.4 Interface utilisateur

L'interface utilisateur (IHM) constitue la partie visible de l'application pour ses utilisateurs. Sa conception a été particulièrement soignée pour garantir une expérience fluide et efficace. L'objectif est de permettre aux utilisateurs d'accomplir leurs tâches rapidement, sans frustration, et de réduire au maximum la courbe d'apprentissage.

Principes d'ergonomie

Plusieurs **principes d'ergonomie** ont guidé la conception de l'interface :

- **Clarté visuelle** : utilisation d'une charte graphique cohérente, hiérarchie claire de l'information, espace blanc suffisant pour faciliter la lecture ;
- **Cohérence** : les éléments d'interface se comportent de la même façon dans toutes les sections de l'application ;
- **Feedback** : chaque action de l'utilisateur déclenche un retour visuel (confirmation, message d'erreur, indicateur de chargement) ;
- **Réversibilité** : les actions destructives nécessitent une confirmation ; les modifications peuvent être annulées ;
- **Accessibilité** : contrastes suffisants, tailles de police adaptées, navigation possible au clavier.

Maquettage préalable

Avant de coder la moindre ligne d'interface, j'ai réalisé plusieurs **maquettes** (wireframes) pour visualiser l'organisation des écrans principaux. Ces maquettes ont été discutées avec mon maître de stage et ajustées en fonction de ses retours et de ceux des utilisateurs cibles. Cette étape de prototypage est essentielle car elle permet d'identifier et de corriger les problèmes ergonomiques avant l'investissement dans le développement.

Écrans principaux

L'application est structurée autour de plusieurs écrans clés qui couvrent les principaux cas d'utilisation :

- **Écran de connexion** : permet à l'utilisateur de s'authentifier via son identifiant et son mot de passe ;
- **Tableau de bord** : page d'accueil affichant les indicateurs principaux et les factures récentes ;
- **Liste des factures** : affichage paginé avec filtres et tri ;
- **Détail d'une facture** : visualisation complète avec accès aux documents joints ;
- **Formulaire de saisie** : création ou modification d'une facture ;
- **Module de recherche avancée** : interface dédiée aux recherches multicritères ;
- **Module d'export** : génération de fichiers CSV ou Excel à partir d'une sélection.

3.5 Tests et validation

La phase de tests revêt une importance capitale dans le développement d'une application destinée à la production. Elle permet de vérifier que le logiciel fonctionne conformément aux attentes et de détecter d'éventuelles régressions lors des évolutions futures.

Stratégie de tests

Plusieurs niveaux de tests ont été mis en œuvre durant le projet :

- **Les tests unitaires** vérifient le bon fonctionnement de chaque composant pris isolément. Une attention particulière a été portée aux méthodes de calcul (TVA, totaux) et aux fonctions de validation.
- **Les tests d'intégration** contrôlent les interactions entre les différents composants, en particulier l'interface avec la base de données.
- **Les tests fonctionnels** simulent les scénarios d'utilisation typiques pour s'assurer que les fonctionnalités répondent aux besoins métiers.
- **Les tests de non-régression** sont rejoués systématiquement après chaque modification pour détecter d'éventuels effets de bord.

Mise en œuvre avec PyTest

Pour automatiser les tests unitaires, j'ai utilisé le framework **PyTest**, qui est l'un des standards de l'écosystème Python. Voici un exemple de test unitaire pour la classe Facture :

```
import pytest
from models.facture import Facture

def test_calcul_ttc():
    """Vérifie le calcul du montant TTC."""
    facture = Facture(montant_ht=100.0, taux_tva=20.0)
    assert facture.montant_ttc == 120.0

def test_calcul_ttc_taux_reduit():
    """Vérifie le calcul TTC avec taux de TVA réduit."""
    facture = Facture(montant_ht=100.0, taux_tva=5.5)
    assert facture.montant_ttc == 105.5

def test_validation_montant_negatif():
    """Le montant HT négatif doit lever une exception."""
    facture = Facture(numero="F001", montant_ht=-50.0)
    with pytest.raises(ValueError):
        facture.valider()

def test_validation_numero_obligatoire():
    """Le numéro de facture doit être renseigné."""
    facture = Facture(montant_ht=100.0)
    with pytest.raises(ValueError):
        facture.valider()
```

Jeux de données de test

Pour mener à bien les tests, j'ai constitué un **jeu de données représentatif** contenant des factures fictives mais réalistes. Ce jeu de données couvre différents cas de figure : montants variables, fournisseurs multiples, dates étalées sur plusieurs années, statuts divers. Il permet de simuler des conditions d'utilisation proches de la réalité.

Pour des raisons de confidentialité, aucune donnée réelle n'a été utilisée à des fins de test. Cette précaution est conforme aux exigences du RGPD et aux bonnes pratiques de sécurité informatique.

Recette utilisateur

Au-delà des tests automatisés, une **phase de recette utilisateur** a été conduite avec des représentants des futurs utilisateurs. Cette phase a permis de :

- Valider l'ergonomie et l'intuitivité de l'interface ;
- Identifier des cas d'usage non anticipés lors de la conception ;
- Recueillir des suggestions d'amélioration ;
- Détecter d'éventuels bugs ou incohérences fonctionnelles.

Cette interaction directe avec les utilisateurs a constitué une expérience particulièrement enrichissante. Elle m'a appris à **écouter activement** les retours, à **argumenter** les choix techniques et à **négoier** les compromis lorsque certaines demandes étaient incompatibles avec les contraintes du projet ou les délais impartis.

Documentation technique

Parallèlement au développement, j'ai produit une **documentation technique** détaillée à destination des futurs mainteneurs de l'application. Cette documentation comprend :

- Une description de l'**architecture générale** de l'application ;
- Le **schéma complet de la base de données** avec descriptions des champs ;
- Les **diagrammes UML** des principales classes ;
- Une **procédure d'installation** pas à pas ;
- Un **guide utilisateur** illustré de captures d'écran.

Cette documentation est essentielle pour garantir la **pérennité du projet**. Elle permet à d'autres développeurs de comprendre rapidement le fonctionnement de l'application et d'intervenir efficacement en cas de besoin.

IV. Bilan du stage

4.1 Compétences techniques acquises

Ce stage a constitué une véritable accélération dans mon apprentissage technique. Si la formation EPSI m'avait fourni les bases théoriques et de premières expériences pratiques, c'est au contact d'un projet réel, en conditions professionnelles, que j'ai pu véritablement consolider et étendre mes compétences.

Maîtrise de Python

Au début du stage, je disposais de connaissances de base en Python acquises durant ma formation. À la fin des cinq semaines, ma maîtrise de ce langage s'est considérablement renforcée. J'ai notamment progressé sur :

- La **programmation orientée objet** avancée (héritage, polymorphisme, méthodes spéciales) ;
- La **gestion des exceptions** et l'écriture de code robuste ;
- L'utilisation de **bibliothèques tierces** (mysql-connector, pandas, openpyxl) ;
- Les **bonnes pratiques** de structuration de projet et de nommage ;
- L'écriture de **tests unitaires** avec PyTest.

Compétences en bases de données

Mon travail avec MySQL m'a permis d'approfondir mes compétences en bases de données relationnelles. J'ai en particulier consolidé :

- La **conception de schémas relationnels** respectant les formes normales ;
- L'écriture de **requêtes SQL avancées** avec jointures, sous-requêtes, agrégations ;
- L'**optimisation des performances** via les index et la réécriture de requêtes ;
- La **gestion des transactions** et la garantie de l'intégrité référentielle ;
- Les **bonnes pratiques de sécurité**, notamment la prévention des injections SQL.

Vision d'ensemble du SI

Au-delà des compétences techniques pointues, ce stage m'a apporté une **vision plus large** de ce qu'est un système d'information d'entreprise. J'ai compris l'importance des interactions entre les composants, des contraintes d'intégration, de la pérennité du code et de la cohérence des choix techniques sur le long terme. Cette dimension stratégique du métier de développeur n'apparaît pas toujours clairement dans un contexte scolaire et constitue un acquis précieux pour la suite de mon parcours.

4.2 Compétences professionnelles

Le stage n'apporte pas que des compétences techniques. Il est aussi un moment privilégié pour développer des compétences professionnelles transversales, parfois appelées **soft skills**, qui sont aujourd'hui indispensables à toute carrière épanouie.

Communication et travail en équipe

L'intégration au sein d'une équipe professionnelle m'a permis de développer ma **capacité de communication**. Que ce soit lors des réunions hebdomadaires, des points individuels avec mon maître de stage ou des échanges avec les utilisateurs métiers, j'ai appris à :

- **Formuler clairement** mes questions et mes besoins ;
- **Reformuler** les explications reçues pour valider ma compréhension ;
- **Adapter mon vocabulaire** selon que je m'adressais à des techniciens ou à des métiers ;
- **Présenter mes travaux** de manière synthétique et accessible ;
- **Accepter la critique constructive** et en tirer parti pour progresser.

Autonomie et organisation

Mon maître de stage m'a accordé une **autonomie progressive** dans la conduite du projet, ce qui m'a obligé à m'organiser et à planifier mon travail de manière rigoureuse. J'ai appris à :

- Découper un projet complexe en **tâches élémentaires** ;
- Estimer la **durée** de chaque tâche et tenir compte des aléas ;
- **Prioriser** en fonction de l'urgence et de l'importance ;
- **Rendre compte** régulièrement de mon avancement ;
- **Alerter** en temps utile en cas de difficulté.

Rigueur et qualité du travail

Le contexte professionnel impose un **niveau d'exigence** nettement supérieur à celui des exercices scolaires. Le code que j'ai produit était destiné à être utilisé en production, à être lu et maintenu par d'autres développeurs, et à s'intégrer dans un environnement existant. Cette réalité m'a poussé à adopter une rigueur constante dans mes développements : noms de variables explicites, commentaires pertinents, gestion systématique des cas d'erreur, tests de mon propre travail avant de le soumettre à validation.

Curiosité et apprentissage continu

Le monde du développement logiciel évolue à une vitesse vertigineuse. Au cours de ce stage, j'ai compris à quel point la **curiosité** et la capacité à **apprendre par soi-même** sont essentielles. Les ressources sont nombreuses (documentation officielle, forums spécialisés, articles techniques) mais il faut savoir les chercher, les évaluer et les utiliser efficacement. Cette compétence d'auto-formation, que j'avais déjà commencé à développer durant mes études, s'est nettement renforcée durant ce stage.

4.3 Difficultés rencontrées

Aucun projet ne se déroule sans accroc, et le mien n'a pas fait exception. Plusieurs difficultés ont émaillé mon stage, mais chacune a été l'occasion d'un apprentissage. Il me semble important d'en rendre compte avec honnêteté, car la capacité à surmonter les obstacles fait partie intégrante du métier.

Découverte de l'environnement technique

Lors de mon arrivée, j'ai été confronté à un environnement technique complexe avec lequel je n'étais pas familier. La configuration des outils, l'accès aux différents systèmes, la compréhension des conventions internes ont demandé un temps d'adaptation significatif. La documentation interne, bien que conséquente, m'a parfois paru difficile à appréhender dans sa globalité durant les premiers jours.

Pour surmonter cette difficulté, j'ai adopté une approche méthodique : prise de notes systématique, questions ciblées à mes collègues, exploration progressive des outils. Cette phase d'acclimatation, bien qu'inconfortable, est inévitable dans tout nouveau poste et constitue en soi un apprentissage précieux.

Compréhension du besoin métier

Comprendre le métier des utilisateurs finaux constitue souvent un défi pour les développeurs. Le vocabulaire spécifique (TVA, taux, échéances, statuts comptables) et la complexité des règles de gestion m'ont demandé un effort de documentation et de questionnement. J'ai parfois dû demander à mes interlocuteurs de répéter ou de reformuler leurs explications, ce qui peut être intimidant lorsqu'on est stagiaire.

J'ai appris que **poser des questions n'est pas un signe de faiblesse, mais au contraire de professionnalisme**. Un développeur qui code sans comprendre le besoin produit inévitablement un résultat insatisfaisant. Cette leçon, simple en apparence, est en réalité fondamentale.

Gestion du temps

Cinq semaines, c'est à la fois beaucoup et peu. Estimer correctement la durée des tâches n'a pas toujours été simple, et certaines fonctionnalités m'ont demandé plus de temps que prévu, notamment celles liées à la recherche multicritères et aux exports. J'ai dû revoir à plusieurs reprises ma planification et accepter de différer certaines évolutions secondaires pour me concentrer sur l'essentiel.

4.4 Apports personnels

Sur le plan personnel, ce stage a été extrêmement riche. Il m'a confirmé mon attrait pour le développement logiciel et m'a donné une vision plus précise du métier auquel j'aspire. J'ai pris confiance en mes capacités et constaté que les compétences acquises en BTS SIO constituent une base solide pour intégrer le monde professionnel.

Ce stage a également renforcé ma **motivation** pour la suite de mon parcours. Je souhaite désormais poursuivre mes études et continuer à développer mes compétences techniques et

méthodologiques pour devenir un développeur accompli et reconnu.

Conclusion

Ce stage de cinq semaines effectué au sein de la **Caisse de Retraite du Personnel Navigant (CRPN)** représente une étape majeure de ma formation en BTS SIO option SLAM. La mission qui m'a été confiée — le développement de l'application **Contrathèque** destinée à centraliser les factures de l'entreprise — m'a offert l'opportunité de mobiliser et de renforcer un large éventail de compétences techniques et professionnelles.

Sur le plan **technique**, j'ai pu mettre en pratique mes connaissances en programmation Python, en conception de bases de données MySQL et en architecture logicielle. J'ai également découvert et adopté de nombreuses bonnes pratiques propres au monde professionnel : gestion des versions, écriture de tests automatisés, documentation technique, sécurisation des données. Ces apports techniques constituent un socle solide sur lequel je pourrai m'appuyer pour la suite de mon parcours.

Sur le plan **professionnel**, ce stage m'a permis de découvrir la réalité du travail en équipe au sein d'une direction des systèmes d'information. J'ai compris l'importance de la communication, de la rigueur, de l'autonomie et de la capacité à rendre compte de son travail. Le contact direct avec les utilisateurs métiers a été particulièrement formateur : il m'a rappelé qu'un logiciel n'est jamais une fin en soi, mais un outil au service de personnes réelles, avec des besoins concrets.

Sur le plan **personnel**, cette immersion a renforcé ma confiance en mes capacités et confirmé mon attrait pour le métier de développeur logiciel. J'ai pris conscience des exigences du monde professionnel mais aussi de l'épanouissement qu'il peut procurer lorsqu'on est passionné par son domaine. L'accueil chaleureux que j'ai reçu à la CRPN et la qualité de l'accompagnement dont j'ai bénéficié ont grandement contribué à cette expérience positive.

Si ce stage a globalement été une réussite, je suis conscient des marges de progression qui s'offrent à moi. Le développement logiciel est un domaine en perpétuelle évolution qui demande une **formation continue** et une **curiosité permanente**. Je souhaite poursuivre dans cette voie en consolidant mes acquis et en explorant de nouveaux horizons techniques : frameworks web modernes, conteneurisation, méthodologies agiles, intégration et déploiement continus.

À l'issue de cette expérience, l'application **Contrathèque** que j'ai contribué à développer constitue une **base fonctionnelle** sur laquelle la CRPN pourra construire des évolutions ultérieures. Les fondations techniques et la documentation que j'ai produites devraient faciliter la reprise du projet par les équipes en place.

Je termine ce rapport avec un sentiment de **fierté** pour le travail accompli et de **gratitude** envers toutes les personnes qui m'ont permis de vivre cette expérience professionnelle stimulante. Ce stage marque une étape importante de mon parcours et constitue, j'en suis convaincu, un tremplin précieux pour la suite de ma carrière dans le développement logiciel.

Glossaire et annexes

Glossaire technique

Terme	Définition
BTS SIO	Brevet de Technicien Supérieur Services Informatiques aux Organisations
SLAM	Solutions Logicielles et Applications Métiers, option du BTS SIO
CRPN	Caisse de Retraite du Personnel Navigant
SGBD	Système de Gestion de Base de Données
MVC	Model-View-Controller, patron de conception logicielle
CRUD	Create, Read, Update, Delete — opérations de base sur les données
SQL	Structured Query Language, langage de requête pour bases relationnelles
RGPD	Règlement Général sur la Protection des Données
IHM	Interface Homme-Machine
API	Application Programming Interface
DSI	Direction des Systèmes d'Information
Git	Système de gestion de versions distribué
PyTest	Framework de tests unitaires pour Python
Injection SQL	Faible de sécurité permettant d'altérer une requête SQL
TVA	Taxe sur la Valeur Ajoutée

Compétences BTS SIO mobilisées

Ce stage m'a permis de mobiliser et de valider plusieurs compétences du référentiel du BTS SIO option SLAM, notamment :

- **Bloc 1** : Support et mise à disposition de services informatiques ;
- **Bloc 2 (SLAM)** : Conception et développement d'applications ;
- **Bloc 3** : Cybersécurité des services informatiques.

Les compétences spécifiques validées incluent notamment : concevoir et développer une solution logicielle, assurer la maintenance corrective et évolutive d'une solution, gérer les données de l'organisation, accompagner les utilisateurs dans la mise en place d'un service,

prendre en compte la sécurité dans le développement.